Zurich University of Applied Sciences

School of Management and Law

Prototyping – Vorlesung 3
SvelteKit Teil 1, JavaScript Grundlagen



Building Competence. Crossing Borders.

Max Meisterhans
max.meisterhans@zhaw.ch

Lernziele

- Ihr könnt mit JavaScript und SvelteKit einfache Webanwendungen erstellen.
- Ihr wisst, wie ihr variable Daten in HTML verwendet (Databinding).
- Ihr könnt mit Svelte Funktionen implementieren und verwenden.
- Ihr könnt mit SvelteKit ein Projekt erstellen.



Was ist JavaScript?

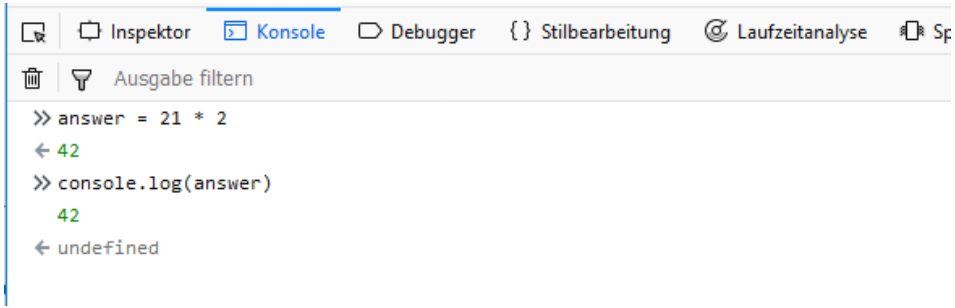
JavaScript („ECMAScript“) …

- ist eine plattformunabhängige Skriptsprache.
- ist objektorientiert.
- läuft typischerweise im Browser.
- wird für die Kommunikation mit Web-Servern benötigt.
- ist erforderlich zur Erstellung von dynamischen (interaktiven) Webseiten.

Running JavaScript using a browser

JavaScript can be executed directly in the browser: Open the developer tools and enter the following code line by line:

```
answer = 21 * 2
console.log(answer)
alert(answer)
```



You can also manipulate the current page. Go to www.zhaw.ch and enter the following code:

```
var logo = "https://upload.wikimedia.org/wikipedia/en/4/48/Batman_Arkham_series_logo.png";
document.getElementById("logo").innerHTML = "<img src=" + logo + ">";
```

JavaScript Frameworks

Frameworks provide generic functionality to build complete applications.

Some popular JavaScript frameworks for building user interfaces are:

- **Svelte / SvelteKit**: An open-source framework for building interactive web applications.
- **Vue.js**: A frontend framework inspired by *Angular.js* and *Reactive.js* for building interactive web applications.
- **Angular**: A *TypeScript*-based open-source front-end web application framework developed by Google. It is used to build complex interactive web applications.
- **React**: A JavaScript framework for web applications maintained by Meta (Facebook). It is also often used to build native applications (with the additional library *React Native*).







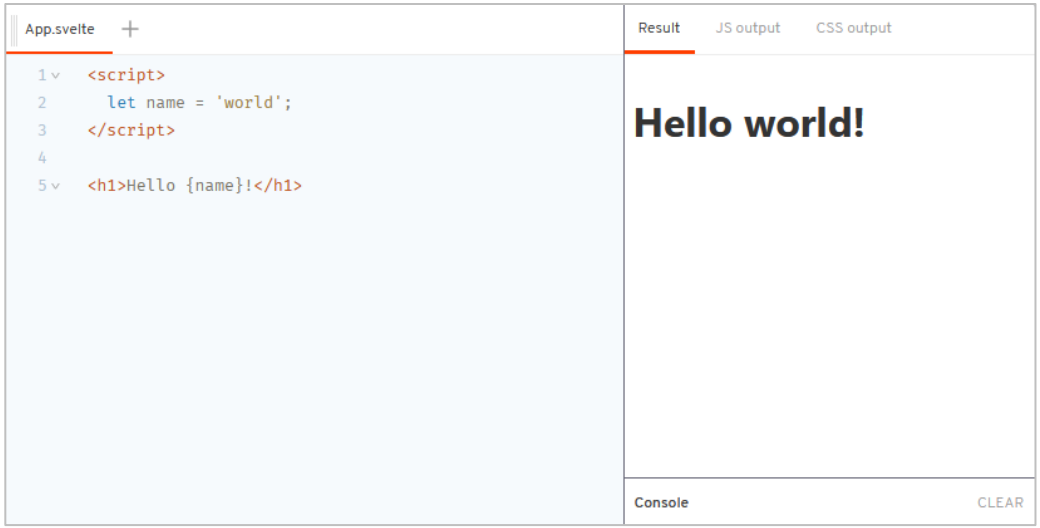


Svelte Introduction

What is Svelte?

***Svelte**, https://svelte.dev/, is an open-source **framework** for building user interfaces*

- Applications and components are defined in *.svelte* files. These files contain HTML, JavaScript and CSS.
- Svelte has its own compiler to convert *.svelte* files to standard JavaScript/CSS code.



Svelte Apps are based on *Components*

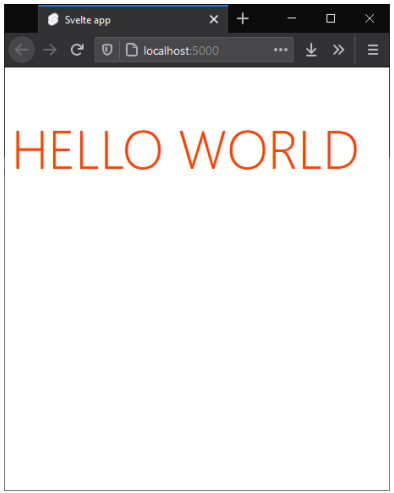
Components are the central units of Svelte.

- A component is a reusable block of code that contains *HTML*, *CSS* and *JavaScript* that belong together.
- A component can be a *complete app*, a *single page*, or just a *custom element*.
- Components can be nested. (We can import components in other components).

```
<script>
    let message = "Hello World";
</script>

<div>
    <h1>{message}</h1>
</div>

<style>
    h1 {
        color: #ff3e00;
        text-transform: uppercase;
        font-size: 4em;
        font-weight: 100;
    }
</style>
```



Svelte components (.svelte files)

JavaScript

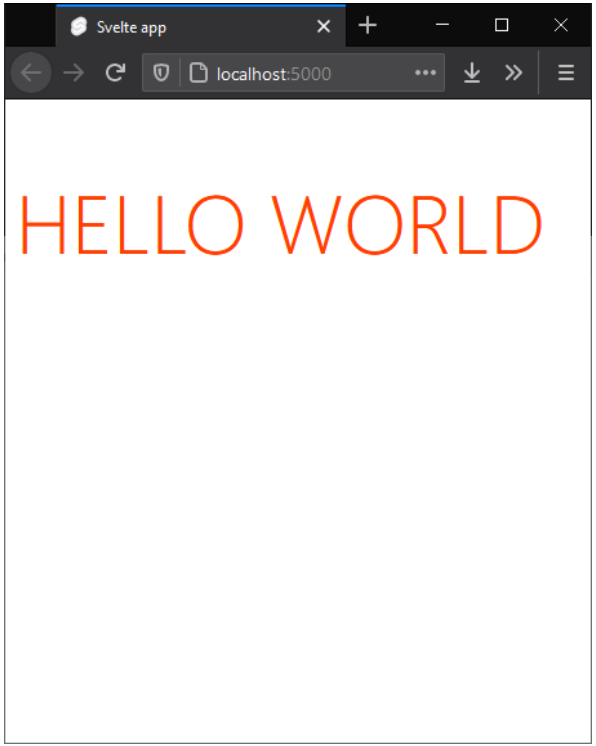
HTML

CSS

```
<script>
    let message = "Hello World";
</script>

<div>
    <h1>{message}</h1>
</div>

<style>
    h1 {
        color: #ff3e00;
        text-transform: uppercase;
        font-size: 4em;
        font-weight: 100;
    }
</style>
```



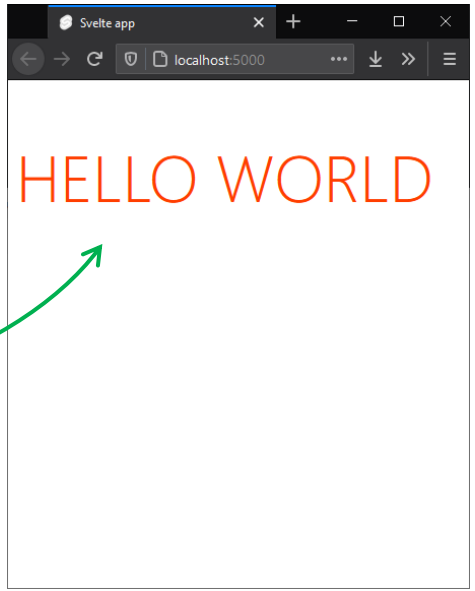
Databinding

Svelte uses curly brackets { .. } to insert JavaScript code, in particular *variables*, into HTML.

```
<script>
    let message = "Hello World";
</script>

<h1>{message}</h1>

<style>
    h1 {
        color: #ff3e00;
        text-transform: uppercase;
        font-size: 4em;
        font-weight: 100;
    }
</style>
```



JavaScript
Variables, Data Types, Operators

Variables

Variables are declared with *var* or *let*, regardless of its data type:

```
// Declaration and initialization with var
var a;
var b = 10;

// Declaration and initialization with var
let c;
let d = 10;
```

Dynamic typing

The data type of a variable is not explicitly set but determined at runtime.

`let`: Block-scoped variable (same as in Java)

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/let

`var`: Function-scoped variable

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/var

Data types in JavaScript

Primitive types of JavaScript:

Data type	Examples	Description
Boolean	`let b = true;`	`True or False`
Null	`let n = null;`	The *intentional* absence of any value.
Undefined	`let u;`	Variables that have no value assigned yet.
Number	`let x = 1;` `let y = 2.54;`	JavaScript does not distinguish between floating point numbers and integers.
String	`let o = "Hello World";` `let p = 'Or with single quotes';`	Text.

... plus *Symbol* and *BigInt.*

Variables and Data Types: Examples

Variables:

```
let age;
age = 39;
let name = "Manfred";
```

No explicit data types:

```
let myAge = 46;      // number
let myName = 'Max'; // string
let imHere = true;  // boolean

// array
let myAnimals = ['Catzilla', 'Harry Pawter'];

// object
let dog = { name: 'Sherlock Bones', breed: 'Dachshund' };
```

Arrays

Arrays can be created with square brackets […]:

```
let myAnimals = ['Catzilla', 'Harry Pawter'];
```

You can access elements by its index:

```
myAnimals[0]; // Catzilla
myAnimals[1]; // Harry Pawter
```

In contrary to Java, the length of an array is not fixed. You can add elements to the end of an array with the method “push”.

More info about manipulating arrays: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array

```
myAnimals.push('Mr. Meowgi');
```

The elements of arrays can have different types. An array can even contain other arrays:

```
let mixed = [1, "two", [true, null]];
```

Operators

Arithmetic operators

```
1 + 2;
5 - 3;
5 * 8;
8 / 5;
8 % 5; // modulo
```

Comparison operators

```
5 === 2 + 4; // Strict equality
5 !== 2 + 3; // Strict-non-equality
10 < 6; // Less than
10 > 20; // Greater than
3 <= 2; // Less than or equal to
5 >= 8; // Greater than or equal to
```

Loops

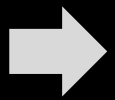
All three variants generate the same output.

```
let candies = ["Mars", "Snickers", "Lion"];

for (let i = 0; i < candies.length; i++) {
    console.log(candies[i]);
}

for (let candy of candies) {
    console.log(candy);
}

let i = 0;
while (i < candies.length){
    console.log(candies[i]);
    i++;
}
```



```
Mars
Snickers
Lion
```

Kurzübung – Loops

Löse folgende Kurzübungen im Svelte Playground: https://svelte.dev/playground

a) Deklariere eine Variable `message` und gib ihr den Wert “Willkommen”

b) Erstelle ein Array mit verschiedenen Namen, z.B. “Max” und “Alex”

c) Gib in einem Loop die `message` und die Namen mit `console.log(...)` aus:

```
Willkommen, Max!
Willkommen, Alex!
```

Conditional statements

The `if` statement is used to check whether a condition is true. Only in this case the code is executed.

Additional conditions can be checked with `else if`:

- The first block for which the condition is fulfilled is executed. The remaining blocks are skipped, even if another condition is fulfilled.
- The `else` block is executed if no other condition is fulfilled.

```
let name = 'Yoda';

if (name === 'Anakin') {
    console.log('I find your lack of faith disturbing.');
} else if (name === 'Yoda') {
    console.log('Do. Or do not. There is no try.');
} else {
    console.log('I am no Jedi.');
}
```

Equality vs. Identity

== is used to check for **equality**.

If the values are not of the same data type, they are converted first.

```
// Equality
let a = 1, b = "1";
a == b; // true
```

=== is used to check for **identity** ("strict equality").

Both values must be of the same data type, otherwise the comparison returns "false".

```
// Identity
let a = 1, b = "1";
a === 1; // true
a === b; // false
```

For relational operators (e.g., <=, >), the operands are first converted to primitives, then to the same type, before comparison.

Logical operators

Logical operators work in the same way as they do in Java:

```
let a = 1;
let b = 7;

console.log(true && false);         // output: false

console.log(true || false);         // output: true

console.log(!(true || false));      // output: false

console.log(a > 0 && b < a);        // output: false

console.log(a > 0 && !(b < a));     // output: true

console.log(a < b || b < 0);        // output: true

console.log(b % 4 === a && a < b); // output: false
```

Kurzübung – Schleifen und Bedingungen

Löse folgende Kurzübungen im Svelte Playground: https://svelte.dev/playground

a) Schreibe ein Programm, das für jede Zahl in einem Array prüft, ob diese durch 3 oder 4 teilbar ist. Beispiel:

```
let zahlen = [5,6,8,12,2];
```

Output via `console.log(...)`:

```
6 ist durch 3 teilbar
8 ist durch 4 teilbar
12 ist durch 3 und durch 4 teilbar
```

b) Erstelle ein Programm, das abhängig von der Uhrzeit den folgenden Text anzeigt:

   a) Zwischen 12:00 und 12:59 → “En Guete”
   b) Sonst, vor 18:00 → “Hallo”
   c) Sonst, wenn 18:00 oder später → “Schöne obig”

Hinweis: Mit JavaScript kannst du wie folgt die aktuelle Stunde abfragen:

```
let datum = new Date();
let stunde = datum.getHours();
```

Functions

Functions

Functions:

- are reusable pieces of code.
- can take input values (parameters).
- return an output.

```
function add(x, y) {
    let result = x + y;
    return result;
}

let myResult = add(1, 2); // myResult is 3
```

Structure of a function:

```
function nameOfFunction(parameter1, parameter2) {
    return parameter1 + parameter1;
}

let result = nameOfFunction(parameter1, parameter2);
```

Using Buttons to call functions (in Svelte)

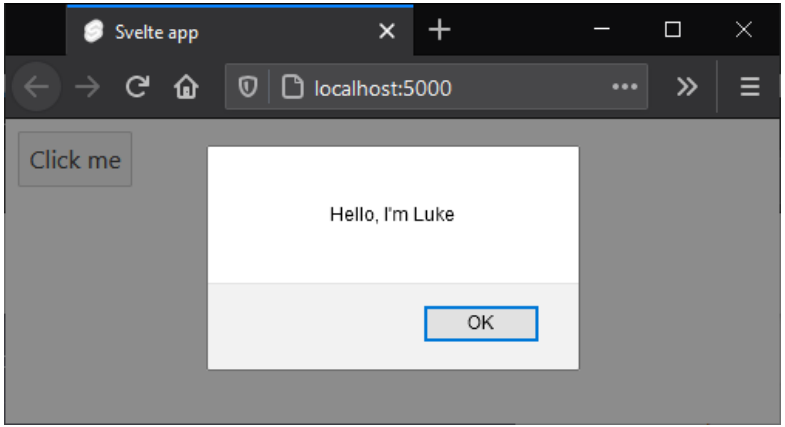
Use HTML events in combination with dynamic attributes to call functions in Svelte:
https://svelte.dev/tutorial/svelte/dynamic-attributes

```
<script>
    let name = "Luke";

    function sayHello() {
        alert("Hello, I'm " + name);
    }
</script>

<button onclick={sayHello}>Click me</button>
```

No brackets!
sayHello~~()~~



Reactivity

Reactivity means that the page will automatically be kept in sync with the application state. In order to make variables reactive, we need to wrap it with `$state(…)`:

https://svelte.dev/tutorial/svelte/state https://svelte.dev/docs/svelte/what-are-runes

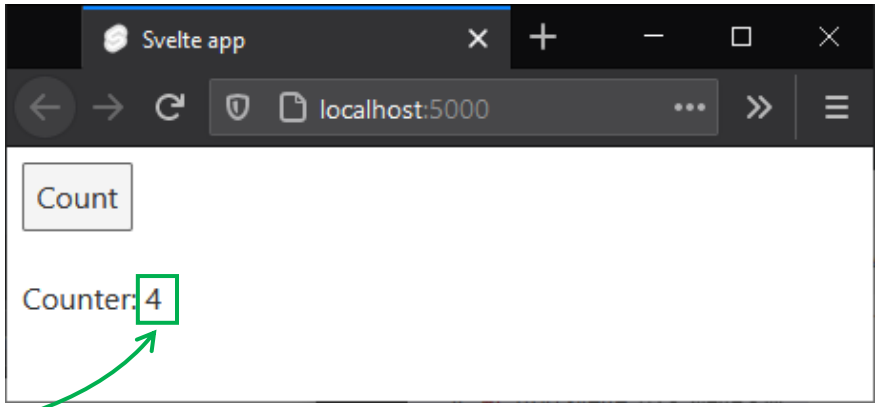
Initial value

```
<script>
    let counter = $state(0);

    function count() {
        counter += 1;
    }
</script>

<button onclick={count}>Count</button>

<p>Counter: {counter}</p>
```



Kurzübung: Funktionen

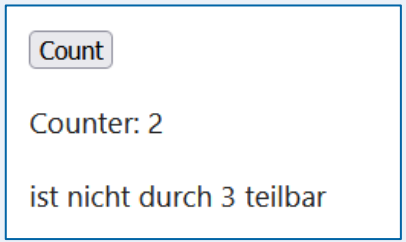
Löse folgende Kurzübungen im Svelte Playground: https://svelte.dev/playground

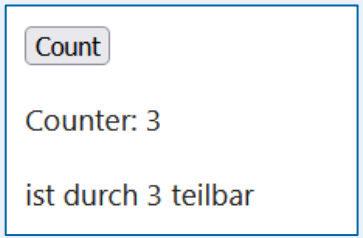
a) Erstelle eine Funktion, die den grössten von drei Werten zurückgibt. Zeige den Rückgabewert via Databinding an.

```
var max = getMax(1,3,2); // max hat den Wert 3
```

Der Code soll mit beliebigen Argumenten (Eingabewerten) funktionieren.

b) Erweitere die App auf der vorherigen Folie, so dass bei jeder Zahl angezeigt wird, ob sie durch 3 teilbar ist.





Projekte mit SvelteKit erstellen

SvelteKit

SvelteKit ist ein Framework für die Entwicklung von Svelte-Apps und bietet unter anderem Routing, Server-side Rendering (SSR) und Performanceoptimierungen. Ausserdem ist es mit Svelte-Kit möglich, Code (z.B. Datenbankzugriffe oder API-Calls) auf den Server auszulagern.

Voraussetzungen:

- **Node.js** (für das Kompilieren von .svelte-Dateien und Development Server)
- **Visual Studio Code** (optional, empfohlen)
- **Svelte for VS Code** Extension für VS Code (optional, empfohlen)
- **Git** (optional, empfohlen)

Zusätzliche Dokumentation:

https://svelte.dev/docs/kit/introduction

https://svelte.dev/docs/kit/creating-a-project

Node.js

Download Node.js:
https://nodejs.org/en/download

Wähle die “LTS” (Long Term Support) Version.



… oder Windows Installer

Projekt erstellen

1. Terminal im Ordner öffnen, in dem das Projekt erstellt werden soll.

   ```
   npx sv create my-app
   ```

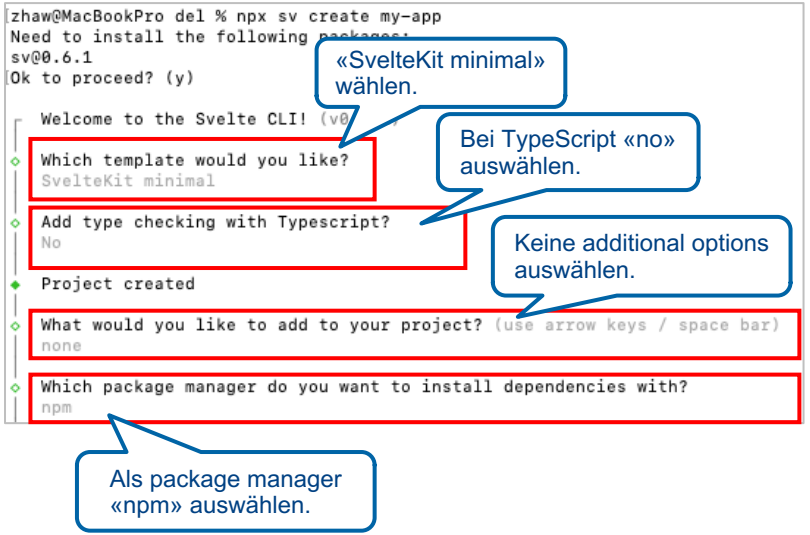
   Name der App. Dieser kann frei gewählt werden. (Keine Leer- oder Sonderzeichen verwenden.)

2. Den in Schritt 1 erstellten Projektordner (z.B. *my-app)* in VS Code öffnen.
   Terminal öffnen und Abhängigkeiten installieren:

   ```
   npm install
   ```

3. Development Server starten

   ```
   npm run dev
   ```



Detaillierte Dokumentation: https://svelte.dev/docs/kit/creating-a-project